

# Autoregressive Inference Before Tools: The Multi-Token Computational Frontier of Tiny Transformers

Paper 0.85 — Multi-Token Inference Research Line

Draft inception — August 2026

## Abstract

A one-token Transformer is a fixed-depth circuit, but a decoder generating multiple tokens is already an iterative machine: each generated token becomes writable state, and subsequent steps can attend to the full trajectory. Before attributing systematic reasoning gains to external agents or tools, we must measure how much computation ordinary autoregressive generation can learn to perform by itself. We introduce a minimal propositional-inference instrument comparing one-token answers with unconstrained and structured multi-token derivations. Training begins with local implication steps and proof chains of length at most two or three; evaluation extends to longer chains, branching, distractors, and unseen proposition identities. We distinguish local-rule acquisition, one-pass internal composition, autoregressive recurrence, termination, and closure-like train-short/test-long extrapolation. The central question is whether a tiny Transformer that reliably learns one implication step can acquire a time-invariant multi-token proof transition and reuse it at depths substantially beyond training, and with what probability across architectures and seeds. This paper supplies the missing baseline between the fixed-model formal computation of Paper 0.7 and the external iterative harness of Paper 0.9.

## 1 Motivation

Suppose the context contains

$$A, \quad A \Rightarrow B, \quad B \Rightarrow C, \quad C \Rightarrow D.$$

A one-token answer to query  $D$ ? requires internal composition before output. An autoregressive model can instead emit

$$A \rightarrow B \rightarrow C \rightarrow D$$

or a structured trajectory such as

$$KNOWN(A), \text{ DERIVE}(B), \text{ DERIVE}(C), \text{ DERIVE}(D), \text{ ANSWER}(TRUE).$$

The model weights are unchanged, but the effective machine is not. Generated tokens form external state:

$$s_{t+1} = F_{\theta}(X, s_{\leq t}).$$

Paper 0.85 isolates this resource before adding tools.

## 2 Scope

The first version uses deliberately simple propositional implication. More complex single-step syntax—negation, conjunction, variables, predicates, functors, frames, algebraic rewrites—should only be added after Paper 0.7 establishes which local operators are reliably learned.

The primary question is therefore:

given a reliably learned local implication step, can autoregression learn reusable proof recurrence?

### 3 Task prerequisite

Every recurrence experiment first passes a local-step gate:

$$A, \quad A \Rightarrow B \vdash B.$$

A model that cannot reliably solve this across held-out proposition symbols cannot support an interpretable closure experiment. Report  $P_{\text{acquire}}^{\text{local}}$  across model and generator seeds.

This gate has not yet been run for Paper 0.85. The current contribution is a CPU-validated symbolic instrument and preregistered accelerator phase. Exact reference-controller traces below validate the task and scoring code; they are not evidence that a Transformer acquired local implication or recurrence.

### 4 Dataset ladder

#### 4.1 R0: direct one-step implication

Use arbitrary proposition symbols and disjoint train/test identities where feasible.

#### 4.2 R1: fixed two-step chains

$$A, \quad A \Rightarrow B, \quad B \Rightarrow C \vdash C.$$

Train both direct-answer and derivation-output conditions.

#### 4.3 R2: variable chain length

Generate

$$A_0, \quad A_0 \Rightarrow A_1, \dots, A_{K-1} \Rightarrow A_K.$$

Train  $K \leq K_{\text{train}}$ , initially  $K_{\text{train}} \in \{2, 3\}$ . Test

$$K \in \{1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64\}.$$

#### 4.4 R3: shuffled rule order

Randomize implication order to remove a trivial left-to-right copying strategy.

#### 4.5 R4: distractor rules

Add irrelevant implications with

$$N \in \{0, 2, 4, 8, 16, 32, 64\}.$$

#### 4.6 R5: branching

Introduce multiple outgoing rules and competing branches only after chain recurrence works. Keep a unique target proof initially, then increase search ambiguity.

#### 4.7 Executable generator and split contract

Each row stores the fact, ordered ground-truth chain, implication multiset, query, minimal proof depth, distractor count, branching factor, shuffle flag, split, and split-specific namespace. Train, validation, and test identities use disjoint prefixes. Branches terminate away from the query, so the target proof remains unique; distractor antecedents are unreachable from the initial fact. The exact validator checks every chain edge, namespace membership, and target-path uniqueness before serialization.

The full CPU materialization contains 2,800 examples. Training is strictly bounded to  $K \in \{1, 2, 3\}$ ; validation and test cover all eleven preregistered depths. All splits cross seven distractor levels, ordered/shuffled presentations, and branching factors one/two. Every row passes the symbolic validator. Four exact target serializations per example produce 11,200 reference traces. These reference outputs pass exact trajectory checks and are labeled `reference_oracle`; they establish generator correctness, not model performance.

## 5 Output-machine conditions

**00: one-token answer.** Emit final conclusion/boolean only.

**01: free multi-token generation.** No prescribed proof grammar; score final answer and trajectory validity.

**02: structured scratchpad.** Use minimal tokens such as

$$\langle STATE : A \rangle \langle DERIVE : B \rangle \cdots \langle ANSWER : D \rangle.$$

No external semantic operation is executed.

**03: teacher-forced derivation training.** Provide correct intermediate proof tokens in training, then autoregress at test. Compare with final-answer-only training where feasible.

## 6 Core recurrence hypothesis

A learned proof procedure should approximate

$$s_t = \text{current known proposition},$$

$$s_{t+1} = \text{NextApplicableRule}(s_t, \mathcal{R}).$$

For a pure chain this transition is local and time-invariant. If acquired, larger test proof depth should primarily increase generated steps rather than require a larger network.

## 7 Internal versus autoregressive composition

Define

$$K_{\text{internal}}(M)$$

as the largest one-token chain length meeting the competence gate, and

$$K_{\text{AR}}(M)$$

as the largest multi-token chain length meeting the same final-answer gate. Primary difference:

$$G_{\text{recurrence}} = K_{\text{AR}} - K_{\text{internal}}.$$

Report full curves, not only maxima.

## 8 Closure signatures

Train on  $K \leq 2$  or 3 and test much larger  $K$ . Evidence for reusable recurrence strengthens when:

1. accuracy remains high far beyond  $K_{\text{train}}$ ;
2. generated step count scales with true proof depth;
3. the same local transition is used at early and late steps;
4. state representations align by proof role rather than absolute generation index;
5. failure depends mainly on accumulated local error or context budget rather than a sharp learned maximum length.

Do not claim literal  $K = \infty$  closure from finite tests.

## 9 Finite-context versus idealized closure

Let  $C_{\text{max}}$  be context capacity. Even a correct recurrence eventually fails if its serialized trajectory exceeds available memory. Report  $K_{\text{AR}}(M, C_{\text{max}})$  and distinguish procedure failure from context overflow.

## 10 Acquisition probability

For architecture  $M$  and training distribution  $\mathcal{D}$  define

$$P_{\text{acquire}}^{\text{recurrence}} = P_s[A(K_{\text{test}} \gg K_{\text{train}}) \geq \gamma].$$

Distinguish recurrence representable but rarely acquired, recurrence reliably acquired, and local rule acquired but recurrence not acquired.

## 11 Architecture grid

Start with

$$L \in \{1, 2, 3, 4\}, \quad W \in \{16, 32, 64\}, \quad H \in \{1, 2, 4\},$$

and training budgets  $T \in \{1, 2, 4\} \times$ . This diagnostic grid, four output conditions, and three model seeds enumerate 1,296 planned accelerator runs. Deeper  $L \in \{6, 8\}$  and wider  $W = 128$  cells are conditional expansions, not part of the initial factorial. Use staged diagnostics: first local competence, then O0–O3 on R1–R2, then expand around the recurrence boundary, then add distractors/branching.

## 12 Mechanistic measurements

At every generation step record next-token distribution, correct-next-state rank/margin, Q/K/V, attention role mass, SA/FF residual increments, representation similarity across recurrence steps, and attention to original rules versus generated state.

For correct next state  $s_{t+1}^*$  define

$$m_t = z_t(s_{t+1}^*) - \max_{y \neq s_{t+1}^*} z_t(y).$$

Test whether  $m_t$  degrades with absolute step index after controlling local-rule difficulty.

## 13 Causal tests

For competent recurrent models:

- patch a same-role state from another proof depth;
- replace an intermediate proposition with correct/incorrect alternatives;
- mask previous proof states while preserving original rules;
- mask original rules after they are supposedly encoded into generated state;
- ablate heads/FF components by recurrence step;
- compare same-role transitions across unseen proposition identities.

A genuine recurrence should be more invariant to absolute step number than a memorized fixed-length trajectory.

## 14 Error accumulation and termination

Estimate conditional transition accuracy

$$p_t = P(s_{t+1} \text{ correct} \mid s_{\leq t} \text{ correct}).$$

Compare whole-proof accuracy with  $\prod_t p_t$  as a simple independent-error baseline. Analyze premature answer, failure to terminate, trajectory drift, wrong applicable rule, and context overflow separately.

Under the deliberately restrictive baseline that transitions are independent conditional on a correct prefix and  $p_t = p$ , whole-proof success is  $p^K$ . The experiment must compare observed success against  $\prod_t p_t$ , because attention to the full generated history permits correlated errors and self-repair. A positive or negative deviation is therefore a diagnostic of dependence, not automatically evidence for or against recurrence.

## 15 CPU-validated serialization frontier

The prompt token count in the pure ordered chain is  $4K + 6$  under the whitespace audit tokenizer. O0 adds one target token, O1 adds  $K + 1$ , and O2/O3 add  $2K + 5$ . At  $K = 64$ , the resulting audited totals are 263, 327, and 395 tokens, respectively. These values provide exact context-budget denominators for the present serialization; tokenizer-specific model runs must also report their true subword lengths.

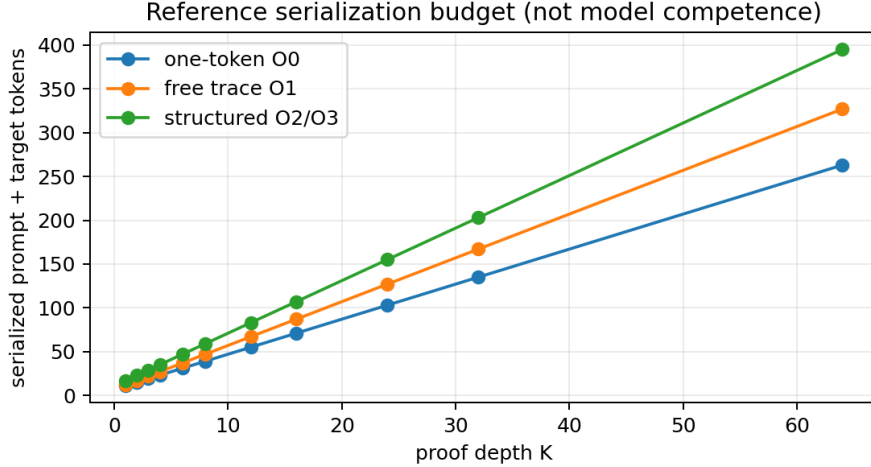


Figure 1: Prompt-plus-target length for the validated reference serialization. This is a dataset geometry result, not a learned competence frontier.

No learned accuracy,  $K_{\text{internal}}$ ,  $K_{\text{AR}}$ , recurrence gain, acquisition probability, termination curve, role invariance, or causal result is currently reported. Those quantities remain blocked on the three-seed R0 accelerator gate. In particular, exact oracle performance is never entered into a model-results table.

## 16 ICL extension

After base recurrence is understood, compare rules learned in weights against novel implication graphs supplied entirely in context with held-out symbols. If the latter succeeds at long depth, the clean interpretation is:

context supplies instance-specific facts/rules + weights supply recurrence procedure.

Do not add this extension before a stable recurrence regime exists.

## 17 Relation to Papers 0.7 and 0.9

Paper 0.7 measures one-pass formal computation. Paper 0.85 adds self-generated intermediate state. Paper 0.9 adds deterministic external operations and an explicit loop. The required ladder is

one-token  $\rightarrow$  free multi-token  $\rightarrow$  structured scratchpad  $\rightarrow$  tool/harness.

Only this can distinguish class expansion from improved learnability/reliability.

## 18 Expected contribution

The strongest positive result would show tiny Transformers trained only on short proofs acquiring a reusable autoregressive transition that extrapolates to much longer chains while one-token performance remains bounded. The strongest negative result would show reliable local implication but bounded or seed-fragile recurrent trajectories. Either outcome locates a computational boundary between fixed-depth composition and explicit external iteration.

## 19 Reproducibility and evidence status

The executable components are:

- generator/evaluator: `src/cl/semantic/autoregressive_proofs.py`;
- phase materialization: `src/cl/experiments/paper085_phase.py`;
- serialization analysis: `src/cl/experiments/paper085_analyze.py`;
- configuration: `configs/paper085/phase_v1.json`.

Machine-readable artifacts are under `docs/papers/paper0_85/results`. The manifest explicitly says that learned-model results are false; each phase-plan row is marked as a planned accelerator run. This status remains until R0 passes at least three model seeds with held-out identities and shortcut controls.

## References

- [1] Z. Li, H. Liu, D. Zhou, and T. Ma. Chain of Thought Empowers Transformers to Solve Inherently Serial Problems. *arXiv:2402.12875*, 2024.
- [2] N. Joshi et al. A Theory of Learning with Autoregressive Chain of Thought. *COLT*, 2025.
- [3] A. A. Bavandpour et al. Lower Bounds for Chain-of-Thought Reasoning in Hard-Attention Transformers. *ICML*, 2025.